

Code-Layout, Readability and Reusability

Date	30 June 2026
Team ID	XXXXXX
Project Name	A Comprehensive Measure of Well-Being
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S. No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	All Python files use 4-space indentation consistently throughout app.py and HumDevIndex.ipynb
2	Proper File Structure	Files and folders are logically organized	Yes	Project is split into Dataset/, Flask/, Flask/templates/, and Training/ folders with each file in correct location
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variables like Development, x_train, y_pred, features_name, input_features clearly describe their purpose
4	Function / Method Names	Functions are descriptively named	Yes	Flask route functions named home(), prediction(), my_home(), predict() clearly indicate their purpose
5	Code Comments	Inline and block comments explain logic	Yes	Every code section has comments like #Importing the libraries, #Train and test split, #Saving our model
6	Modular Design	Code is split into reusable functions/modules	Yes	Training code is in separate notebook, Flask backend in app.py, and UI in separate HTML template files

7	No Redundant Code	Duplicate or unused code is removed	Yes	No duplicate imports or unused variables. Each function serves a single clear purpose
8	Error Handling	Exceptions and errors are handled gracefully	Partial	Flask debug=True handles basic errors. Out-of-range predictions show a custom message on result page

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	HDI.pkl (Trained Model)	Python / Pickle	Loaded in app.py for every prediction request without retraining	High
2	predict() function	Python / Flask	Called every time user submits the prediction form via /predict POST route	High
3	features_name list	Python	Reused in both Jupyter notebook training and Flask app.py for consistent column naming	High
4	resultnew.html template	HTML / Jinja2	Reused for all 4 prediction categories — Low, Medium, High, Very High HDI	High
5	X.fillna(X.mean())	Python / Pandas	Applied to handle missing values — can be reused for any similar HDI dataset	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Project is well organized into separate folders for dataset, training, Flask backend, and HTML templates
Readability	5	All variables, functions, and routes have clear descriptive names with inline comments throughout
Reusability	4	HDI.pkl model and Flask routes are fully reusable. HTML templates use Jinja2 for dynamic content
Documentation / Comments	5	Every major code section has comments explaining what it does and why
Overall Score	4	Clean, readable, well-structured project with modular design and good code practices